Программирование в среде

U N I T Y

на языке C#

Структура программы в Unity на C#

Сценарий **–** это компонент **Script** объекта в среде Unity, которым определяется его поведения на основе созданного скрипта – программного кода. Сценарии в Unity представляют собой компоненты, наследующие от класса ***MonoBehaviour*** – базового класса компонентов-сценариев.

Исходная структура компоненты **Script** с именем **name1:**

```
using UnityEngine;
using System Collections;
using System.Collections.Generic;
public class name1 Script : MonoBehaviour {
  void Start ()
  {  …  }
  void Update ()
  {  …  }
}
```

Класс MonoBehaviour

UnityEngine – библиотека, созданная разработчиками Unity, содержит класс **MonoBehaviour,** который требуется для взаимодействия C#-кода с Unity-функциями.

Функция **Start() {…}**

определяет код, который выполняется при запуске приложения (выполняется только один раз для инициализации переменных)

Функция **Update() {…}**

определяет код, перерисовывающий экран 50 раз в секунду (по умолчанию), т. е. создает программную анимацию на экране, если в этой функции задан шаг изменения какого-либо параметра объекта на сцене.

Базовые понятия в сценарии Unity

Все компоненты сценариев **Script** наследуются от класса **MonoBehaviour.**

Все объекты сцены могут представлять собой экземпляры класса **GameObject,** основным назначением которого является обеспечение объекта добавленной к нему компонентой со сценарием **Script**, наследуемым от базового класса **MonoBehaviour.**

В Unity объект **GameObject** представляет собой контейнер, у которого есть программный код на C#, хранящийся в компоненте **Script,** а также набор других компонент, от которых зависит вид и поведение объекта на сцене.

*Для редактирования кода двойным щелчком мыши по компоненте **Script** объекта вызывается встроенный редактор **MonoDevelop** либо общий редактор **VisualStudio**.*

Изменение координат объекта

Запись обращения к свойству позиции ***position*** объекта в пространстве (как и любому другому свойству объекта):

gameObject.GetComponent<Transform>().position

Для сокращения записи обращения к компоненте **<Transform>** используется свойство **transform** для игрового объекта (аналогично для поворота ***rotation*** и масштаба ***localScale***):

transform.position

Для хранения координат объекта в 3D-пространстве используется тип данных **Vector3**, имеющий 3 параметра, соответствующих значению координат по осям **X, Y, Z**

Например, определив переменную **pos** типа **Vector3**

Vector3 pos = transform.position;

получаем доступ к координатам объекта по осям:

pos.x, pos.y, pos.z

соответственно **X, Y** и **Z**

Поступательное движение объекта

Для измерения расстояния Unity использует <u>условные единицы</u>.

Пример кода для организации непрерывного движения по оси **X** можно использовать следующий код в теле функции **Update()**:

```
void Update() {
Vector3 pos=transform.position;
Vector3 npos=new Vector3(pos.x+0.1f,pos.y,pos.z);
transform.position=npos;  }
```

Можно выполнить то же действие и с помощью более компактного кода в функции **Update()**:

```
Vector3 pos=transform.position;
pos.x+=0.1f;
transform.position=pos;
```

Поступательное движение объекта

То же непрерывное перемещение объекта можно записать еще более компактно в одну строку:

```
void Update ()  {
transform.position += new Vector3(0.1f, 0, 0); }
```

Для создания непрерывного перемещения объекта удобно использовать функцию **Translate(a,b,c)**, аргументы которой задают ***перемещение*** объекта соответственно по осям **X, Y, Z.**

Например, тот же эффект получим записав следующий код:

```
void Update () {
transform.Translate(0.1f, 0, 0); }
```

А для того чтобы перемещение происходило не непрерывно, а только по нажатию клавиши, например, **Q** необходимо в **Update()** использовать конструкцию **Input.GetKeyDown (KeyCode.Q)** с условным оператором **if**:

```
if (Input.GetKeyDown (KeyCode.Q)) { transform.Translate(0.1f, 0, 0); }
```

Динамическое масштабирование объекта

Для программного масштабирования объекта используется свойство **localScale.** При этом учитывается, что хотя в компоненте объекта **Transform** есть свойство **Scale**, но оно является глобальным параметром для отображения объекта на сцене, а масштабирование объекта должно производиться в его локальных координатах.

Например, для непрерывного увеличения размеров объекта по оси **X** можно использовать следующий код:

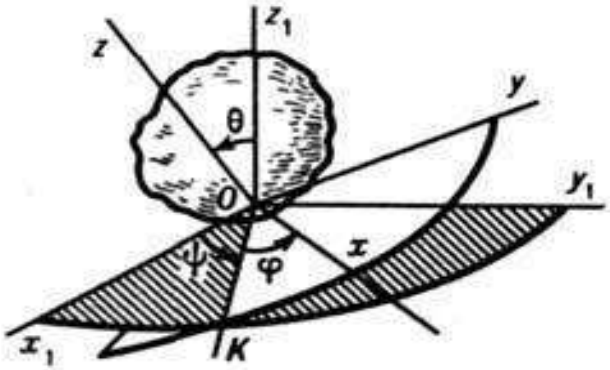
```
void Update ()
{  transform.localScale += new Vector3(0.1f, 0, 0); }
```

Следующий программный код будет масшабировать объект одновременно по осям **Y** (уменьшать на 0.5 усл. ед.) и **Z** (увеличивать на 2 усл. ед.) при каждом нажатии на клавишу **Q**:

```
void Update ()
{ if (Input.GetKeyDown (KeyCode.Q)) {
    transform.localScale += new Vector3(0, -0.5f, 2); }  }
```

Повороты в 3D-пространстве

Углы Эйлера – это три угла поворота объекта относительно осей в локальной системе координат *XYZ*, которые позволяют привести любое начальное положение объекта к заданному.



Углы Эйлера φ, ψ, θ для поворота задаются при обращении к свойству ***eulerAngles*** компоненты ***transform***:

transform.eulerAngles=new Vector3(a1,a2,a3);

Углы поворота задаются через значения аргументов вектора ***Vector3***, которые совпадают с углами поворота объекта соответственно вокруг осей *X, Y* и *Z*. При этом численно реальные углы Эйлера φ, ψ, θ для поворота не совпадают с углами поворота *a1, a2, a3* в векторе ***Vector3***, определяемыми в свойстве ***Rotation*** соответственно по осям *X, Y, Z* в прямоугольной системе координат для компоненты ***Transform*** в окне ***Inspector***.

Вращение объекта

Для задания углов поворота в Unity используются <u>градусы</u>

Для поворота в 3D-пространстве используются ***углы Эйлера***, задаваемые в свойстве ***eulerAngles*** для компоненты **transform**:

```
void Update () {
if (Input.GetKey (KeyCode.A)) {
        transform.eulerAngles = new Vector3 (10, 30, 20); }
}
```

Для создания вращения удобно использовать функцию **Rotate(a,b,c)**, аргументы которой задают ***приращения угла поворота*** объекта вокруг соответственно осей **X, Y, Z.** Например:

```
void Update () { transform.Rotate(1, 2, 3); }
```

Здесь задается непрерывное вращение объекта с приращением в каждом кадре угла поворота вокруг осей: **X** на 1**, Y** на 2**, Z** на 3 градуса.

Вращение с помощью кватернионов

В поворотах с помощью функции **Rotate()** или свойства transform **eulerAngles** необходимо задавать в общем случае ***три угла поворота*** вокруг осей **X, Y** и **Z**, при этом обратное вращение для них будет некоммутативной (неперестановочной) операцией.

При задании вращения с помощью **кватерниона (Quaternion)** задается ***один угол поворота*** и ***ось вращения*** вдоль вектора ***Vector3(a,b,c)*** в трехмерном пространстве. Такое вращение позволяет полностью восстановить первоначальное положение объекта после его поворота, задав угол поворота с обратным знаком при этом оставив ось поворота прежней.

Кроме того, с помощью кватернионов можно осуществить более гладкую интерполяцию между двумя разными положениями объекта, чем это делают углы **Эйлера** или функция **Rotate**.

Кватернион можно представить конструкцией, зависящей от четверки параметров **q=(a,x,y,z),** обычно задаваемых в виде **q=[a,v]**, где: **a** – число, а **v=v(x,y,z)** – вектор в трехмерном пространстве.

Поворот с использованием кватернионов

В общей форме вращение объекта с помощью кватерниона задается следующим образом:

```
Quaternion orig;
void Start()
   {  orig = transform.rotation;  }
void Update()
   {Quaternion rot1 = Quaternion.AngleAxis(5, new Vector3(1,2,3);
     transform.rotation = orig * rot1;
   }
```

Такое же вращение можно запрограммировать и короче:

```
void Update()
   { Quaternion rot1 = Quaternion.AngleAxis(5, new Vector3(1,2,3);
     transform.rotation *= rot1;  }
```

Использование ключевых векторов

Если вращение объекта должно происходить вокруг определенной оси координат **X, Y** или **Z**, то для упрощения записи можно использовать ключевые векторы осей: ***Vector3.up –*** ось **Y,** ***Vector3.right*** *– ось* ***X*** и ***Vector3.forward*** *– ось* ***Z.***

Например, для вращения вокруг вертикальной оси :

```
void Update(){
 Quaternion rot1 = Quaternion.AngleAxis(3, Vector3.up);
 transform.rotation *= rot1; }
```

При этом конструкция ***Vector3.up*** означает поворот вокруг оси **Y**, вместо нее можно было бы записать **new Vector3(0,1,0).**

Однако в этом случае (при вращении объекта вокруг определенной оси координат) нет необходимости использовать достаточно сложную конструкцию кватерниона, а с тем же успехом использовать более простую функцию **Rotate()**:

```
void Update(){    Rotate(0,3,0)     }
```

Конструкция Input.GetAxis

Конструкция **Input.GetAxis()** возвращает число с плавающей запятой, которое используется для задания перемещения объекта клавишами клавиатуры в горизонтальной плоскости на сцене (влево-вправо и удаление-приближение) или его вращения вокруг вертикальной и горизонтальной оси координат соответствующими движениями курсора мыши по экрану.

Конструкция в аргументе использует ***именные константы***:

- **Horizontal** для клавиш **A, D** или «стрелки» **<, >**;
- **Vertical** для клавиш **W, S** или «стрелки» **вверх**, **вниз>**;
- **Mouse X** для движения курсора мыши по горизонтали влево-вправо;
- **Mouse Y** для движения курсора мыши по веритикали вверх-вниз.

Настроить «чувствительность» реакции на нажатия клавиш и перемещения курсора мыши по экрану можно в окне, открывающемся командой меню: **Edit/Project Settings/Input.**

Перемещения с помощью Input.GetAxis

Для вращения объекта вокруг осей **X, Y** при движении курсора мыш по экрану можно записать следующий программный код:

```
void Update() {
float a = Input.GetAxis ("Mouse Y"); //движение мыши по вертикали
float b = Input.GetAxis ("Mouse X"); //движение мыши по горизонтали
transform.Rotate (a, b, 0);  }
```

Для перемещения объекта по осям **X**, **Z** клавишами на клавиатуре можно записать следующий программный код:

```
void Update() {
float c = Input.GetAxis ("Horizontal"); //клавиши: A, D (стрелки: <, >)
float d = Input.GetAxis ("Vertical"); //клавиши: W, S (стрелки: ^, вниз)
transform.Translate (c, 0, d);  }
```

Здесь функция **Translate()** выполняет те же действия, что и функция **Rotate()**, но только для перемещения по осям координат.

Необходимо учитывать, что все получаемые в конструкции **Input.GetAxis()** числовые значения **a, b, c, d** можно, вообще говоря, использовать для выполнения любых соответстующих операций в программном коде.